

Structured Outputs, Function Calling and Tool Use

Topics

- e! JSON mode and structured response schemas
- e! Validating outputs with Pydantic
- e! Function calling: defining tools and routing user intent
- e! Designing safe tool input schemas
- e! Reliable structured data extraction
- e! Handling malformed or partial model output"

Learning Objectives

By the end of this lesson, you will be able to:

- e! Generate predictable outputs using JSON mode and response schemas.
- e! Validate structured model responses with Pydantic.
- e! Define tools and route user requests through function calling.
- e! Design safe tool schemas with clear constraints and permissions.
- e! Extract reliable structured data from unstructured text.
- e! Handle malformed, incomplete, or invalid model output.

JSON Mode and Structured Response Schemas

Why Structured Output Matters?

If **code** consumes the answer, free text is a liability. A schema makes it **directly usable**.

Free text — hard to parse

"Sure! The order total is about \$42, shipping to Berlin, and it looks like it's arriving next Tuesday I think."

Regex-fragile · Ambiguous · Breaks on Rephrase

Schema JSON, code-ready

```
Schema JSON
{ "total": 42.00, "city": "Berlin", "eta": "2025-11-18" }
```

Typed fields · Validates · Drops into objects

JSON Mode: Constrain the Output Format

JSON mode forces the reply into **valid, schema-shaped JSON**. It is about format only, no tool is invoked.



Prompt + schema
"return JSON like..."



LLM · JSON mode
Must emit valid JSON



Validated JSON
Matches the schema



App objects
parsed and typed

Best Practice: Still validate against your schema in code — JSON mode guarantees valid JSON, not the right fields.

Response Schema: Define, Call and Parse

A response schema lets you define the output shape once, call the model against it, and parse the result into a typed object automatically.



Schema

Defines the exact fields and types expected back.



JSON-mode call

Passes the schema in; forces a matching JSON reply.



Typed object

Parses JSON into a real object fields ready to use.

```
Schema
class RAGResponse(BaseModel):
    answer: str
    sources: list[str]
    confidence: float
```

```
JSON-mode call
resp = client.chat(
    response_format=RAGResponse,
    messages=msgs
)
```

```
Typed object
obj.answer      # str
obj.sources     # list[str]
obj.confidence  # 0.92
```

Validating Outputs with Pydantic

Why Validate Outputs with Pydantic?

Pydantic acts as a gate between model output and application logic

Model Output



May be valid JSON but still contain wrong fields, missing values, or bad types.

Pydantic Model



Defines the required shape using Python types and constraints.

Accepted Data



Only validated objects move forward to storage, APIs, or tools.

Validation catches errors early: Before a refund is created, a database is updated, or a message is sent.

Pydantic Validation Workflow

The model response is treated as input data, not as trusted truth.

Raw JSON
From LLM

BaseModel
Schema class

Validate
Types + rules

Object
Use safely

Common checks

- Required keys exist
- Values match expected data types
- Enums stay within allowed labels
- Numbers and strings follow constraints

Common failures

- Missing order_id for an order tool
- priority returned as number instead of label
- Invalid label such as "very angry"
- Empty field where a value is required

Validation Errors Become Feedback

A failed schema check should trigger a controlled recovery path

Validation Fails

Missing or wrong data

Error Message

Specific field issue

Repair Prompt

Ask model to fix only JSON

Fallback

Clarify or review

Bad Pattern



Ignore the error and let business logic guess what the model meant.

Better Pattern



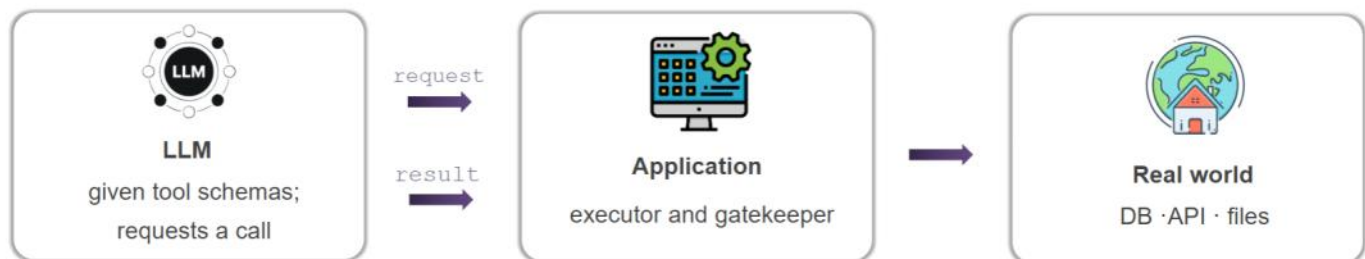
Use the validation error to repair, retry, ask the user, or route to human review.

Demonstration 1: Generating JSON-Structured Outputs from an LLM

Function Calling: Defining Tools and Routing User Intent

Function Calling: Model Requests, App Executes

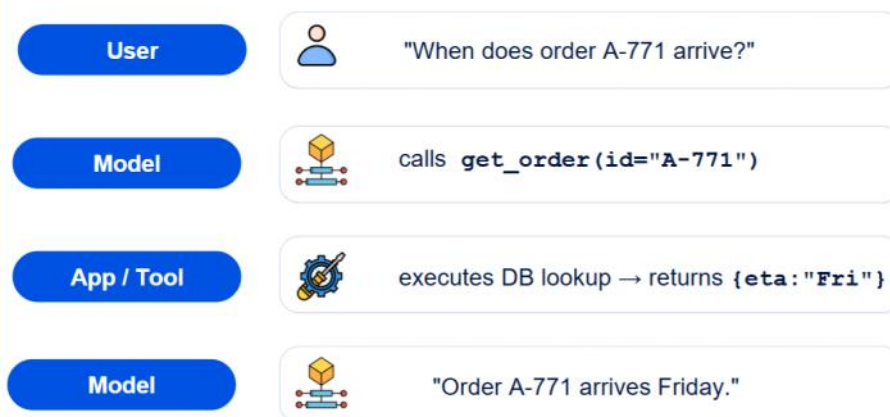
The model **chooses** a tool and emits `name + arguments`. Your **application** runs the real function.



Never draw the model running code. It only produces a call request; the application decides whether and how to execute it.

Function Calling Round-Trip

Three lanes, four hops: the model requests, the app executes, the model answers.



Defining a Tool Schema

Four fields describe a callable tool. The **description** is what the model reads to decide when to use it.



name

Unique Identifier



description

when and why to call drives routing



parameters

Names + types (JSON schema)



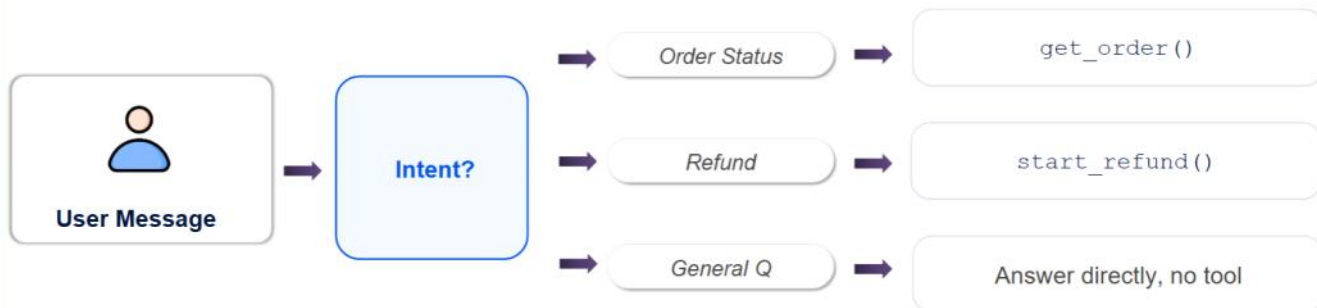
required

which params are mandatory

```
json
{
  "name": "get_order",
  "description": "Look up an order's status by its id",
  "parameters": {
    "type": "object",
    "properties": {
      "id": { "type": "string" }
    },
    "required": ["id"]
  }
}
```

Routing User Intent to the Right Tool

The model classifies intent and **branches** to the matching tool, or answers directly.



Best Practice: write tool descriptions like routing rules. Precise triggers reduce wrong-tool and unnecessary calls.

Designing Safe Tool Input Schemas

Safe Tool Input Schemas Narrow the Action Space

A safe schema tells the model what it is allowed to request. It should not expose open-ended commands or hidden power.

```
Risky

run_backend_task(command: string)

# accepts anything
# unclear permission
# difficult to audit
```



```
Safer

get_order_status(order_id: string)
create_refund(order_id, reason)

# narrow action
# typed fields
# policy checks
```

Schema Constraints Are Guardrails

Use field types, enums, ranges, and confirmation flags to limit unsafe execution

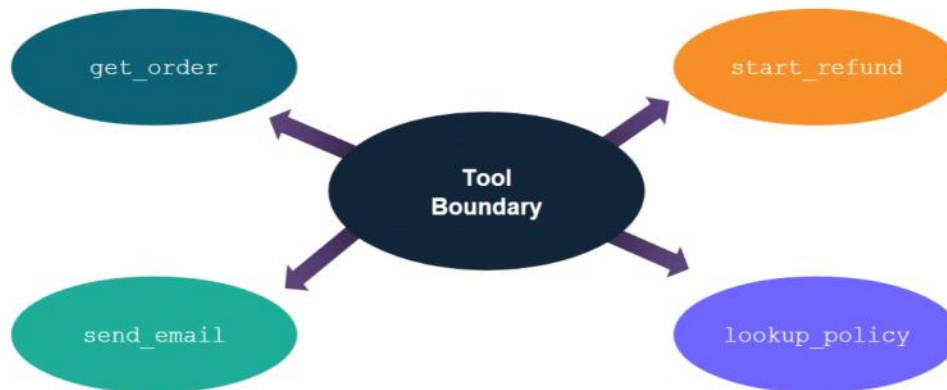
Tool call must pass every gate before execution.



Example: refund_amount must be a number, within policy range, and require explicit confirmation for irreversible actions.

Small Tools, Clear Boundaries

One broad tool is convenient, but dangerous. Multiple focused tools are easier to validate, authorize, test, and log.



Best Practice: write tool descriptions like routing rules. Precise triggers reduce wrong-tool and unnecessary calls.

Reliable Structured Data Extraction

Reliable Structured Data Extraction

Extraction converts real-world text into fields that can be searched, routed, stored, audited, or passed to tools.

Customer email
"I ordered headphones last week. Tracking says delayed and I want a refund."

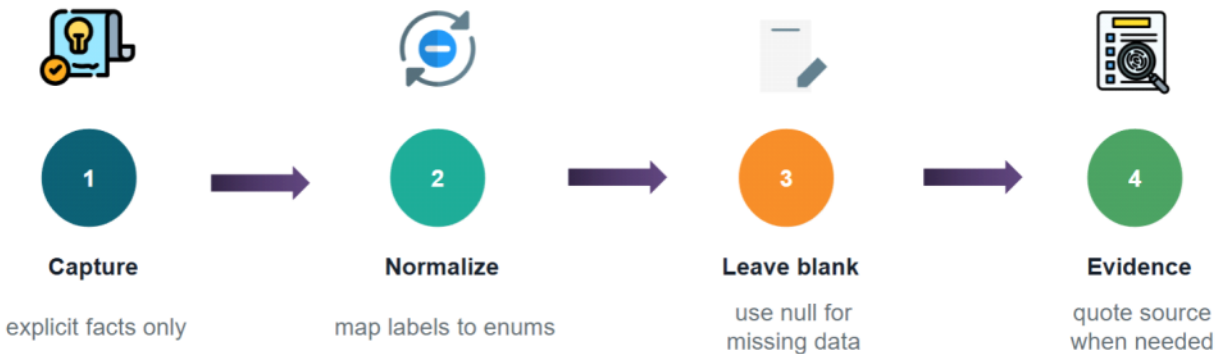


```
{
  "product": "headphones",
  "intent": "refund",
  "issue": "delayed_delivery",
  "order_id": null
}
```

Do not invent missing values. Use null and ask for clarification

Extraction Rules Reduce Hallucination

Define what to capture, what to ignore, and what to leave blank

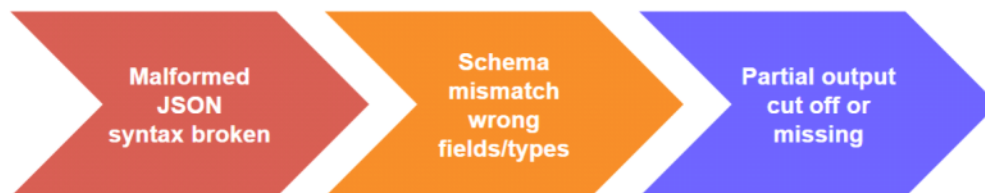


For extraction tasks, accuracy improves when the model is told that "not found" is an acceptable answer

Handling Malformed or Partial Model Output

Malformed vs. Partial Output

Different failures need different recovery paths. First classify the failure, then repair, retry, clarify, or escalate..



Do not continue with broken data. Detect the failure type and use a controlled recovery strategy

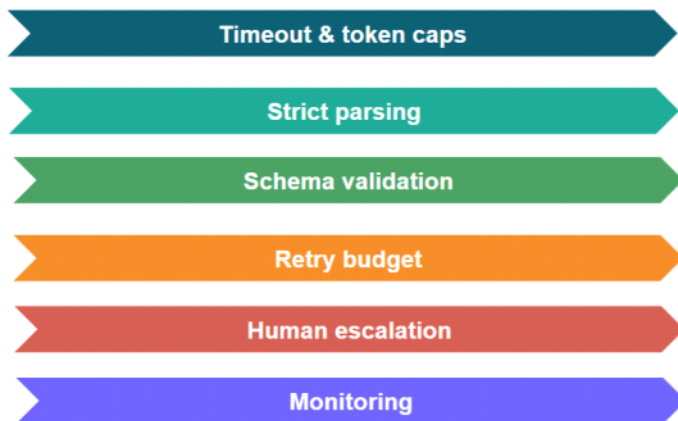
Recovery Workflow for Bad Output



```
Repair prompt

Return only corrected JSON.
Preserve valid values.
Use null for missing information.
Do not change field meanings.
```

Production Patterns for Robust Outputs



Layered defenses make model failures observable and recoverable..

Track error rates by prompt version, model, schema, and input type. Use logs to improve the next release.

Every response is untrusted until it passes parse, schema, and policy checks.

End-to-End Reliable Tool Workflow



What makes it reliable?

- Schema contracts define expected data
- Pydantic catches wrong or missing fields
- Tool schemas limit action space
- Recovery paths handle broken output
- Logs and tests improve each releases

Demonstration 2: Handling Malformed or Partial Model Output

Summary

In this lesson, you have learned to:

- Structure LLM responses using JSON and schemas.
- Validate fields, types, and constraints with Pydantic.
- Use function calling to connect models with external tools.
- Apply schema guardrails before executing tool requests.
- Extract accurate data without inventing missing values.
- Recover from malformed or partial outputs using retries and fallbacks.